LLM stance measurement under a two-stage stratified sample

Table of contents


```
# ----------------------------------------------------------------------------
# CONFIGURATION (the only block you edit)
# ----------------------------------------------------------------------------
st <- list(
  seed            = 2026L,
  in_path         = "outputs/paragraph_theta.parquet",  # Script 1's handoff
      topics_csv                      =    "outputs/topics_meta.csv",       #
topic,label,description,proposition
  out_dir         = "outputs_stance",
  simulated       = TRUE,            # THE one-flag swap; FALSE at work

  selected_topics = NULL,            # e.g. c(1L, 3L, 6L); NULL only in
                                     # simulated mode (top 3 by theta mass)
  topic_col       = NULL,            # name of an existing dominant-topic
                                     # column in the parquet, if you saved one
                                     # in Script 1; NULL derives it here from
                                     # the theta columns (one max.col line)
  min_theta       = 0,               # optional scope floor: drop paragraphs
                                     # whose dominant theta is below this
                                     # (0 keeps everything)

  # --- budget (reporting only; nothing is sampled) --------------------------
  calls_per_window = 300L,          # quota: calls per 4-hour window PER MODEL
  era_width        = 5L,            # Year grouping for the era tabulation

  # --- LLM ------------------------------------------------------------------
  # Persona: analyst-supplied framing prepended to the system prompt (may be
  # ""). Keep it neutral; it is part of the measurement record.
  persona          = "You are a nonpartisan public-comment analyst.",
  # >= 2 models so agreement is measurable. OpenRouter slugs for now; the
```

```
  # second is a PLACEHOLDER: pick a slug from the OpenRouter catalog. At work,
  # replace with the endpoint's model strings (no openrouter/ prefix).
  models           = c(maverick = "openrouter/meta-llama/llama-4-maverick",
                       second   = "openrouter/CHANGE-ME"),
  retries          = 1L,            # PRODUCTION: 16L
  retry_wait       = 5L,            # PRODUCTION: 30L * 60L

  n_gold           = 60L            # blind hand-coding subsample size
)

suppressPackageStartupMessages({
  library(arrow)      # parquet
  library(tidyverse)  # LAST so dplyr verbs win
})
# Namespaced only: ellmer (LLM calls), irr (kappa), rlang (hash).
theme_set(theme_minimal(base_size = 12))
dir.create(st$out_dir, showWarnings = FALSE)

cache_rds <- function(path, expr) {
  if (file.exists(path)) return(readRDS(path))
  x <- force(expr); saveRDS(x, path); x
}
cache_parquet <- function(path, expr) {
  if (file.exists(path)) return(arrow::read_parquet(path))
  x <- force(expr); arrow::write_parquet(x, path); x
}
p_out <- function(f) file.path(st$out_dir, f)

source("llm_stance_source.R")
```

Inputs: the paragraph corpus and the analyst's topic file

What this does: reads the paragraph parquet, validates the input contract, and loads the topic metadata CSV that drives everything topic-related. Why a CSV: the labels and descriptions start as AI drafts from Script 1, but the file is the analyst's, edited freely, and the **proposition** column is written by the analyst alone since it is the measurement target. Setting `st$selected_topics = c(1, 3, 6)` subsets both the corpus and this lookup; every table, plot, and prompt then uses the CSV's label and proposition for those topics. In simulated mode a placeholder CSV is written once so the required format is visible on disk.

```
para_theta <- arrow::read_parquet(st$in_path)
theta_cols <- str_subset(names(para_theta), "^topic_\\d+$")
K <- length(theta_cols)
stopifnot(
  "input must carry topic_1..topic_K"                  = K > 0,
  "input must carry para_uid, atom_id, para_id, Year, n_tokens, text" =
    all(c("para_uid", "atom_id", "para_id", "Year", "n_tokens", "text") %in%
```

```
          names(para_theta))
)

sel <- st$selected_topics
if (is.null(sel)) {
  if (!st$simulated)
    stop("st$selected_topics is NULL. Set the topic numbers to measure.")
  sel <- para_theta |>
    dplyr::select(dplyr::all_of(theta_cols)) |>
    colSums() |> sort(decreasing = TRUE) |> names() |>
    str_remove("topic_") |> as.integer() |> head(3)
  cat("PLACEHOLDER selected topics (simulated mode):",
      str_c(sel, collapse = ", "), "\n")
}
```

```
PLACEHOLDER selected topics (simulated mode): 3, 5, 4
```

```
stopifnot("selected topics must exist in the model" = all(sel <= K))

if (!file.exists(st$topics_csv)) {
  if (!st$simulated)
    stop("Topic file not found: ", st$topics_csv,
         ". Provide a CSV with columns topic, label, description, proposition.")
  tibble(topic       = sel,
         label       = str_c("topic ", sel, " (PLACEHOLDER)"),
         description = "placeholder description",
         proposition = str_c("the policy direction of topic ", sel,
                             " (PLACEHOLDER)")) |>
    readr::write_csv(st$topics_csv)
  cat("PLACEHOLDER topic CSV written to", st$topics_csv,
      "so the required format is on disk.\n")
}
```

```
PLACEHOLDER topic CSV written to outputs/topics_meta.csv so the required format
is on disk.
```

```
topics_meta <- readr::read_csv(st$topics_csv, show_col_types = FALSE)
stopifnot(
  "topic CSV needs columns topic, label, description, proposition" =
    all(c("topic", "label", "description", "proposition") %in%
          names(topics_meta)),
  "every selected topic needs a CSV row" = all(sel %in% topics_meta$topic)
)
topics_meta <- topics_meta |> dplyr::filter(topic %in% sel)
```

```
if (!st$simulated && any(str_detect(st$models, "CHANGE-ME")))
  stop("st$models still contains the CHANGE-ME placeholder. Pick a real ",
       "model string before a production run.")
if (any(is.na(topics_meta$proposition) |
        !nzchar(str_squish(topics_meta$proposition))))
  stop("Empty proposition for at least one selected topic. Stance has no ",
       "meaning without a stated target; write one per topic in the CSV.")
knitr::kable(topics_meta, caption = "Selected topics: the analyst-edited lookup
driving strata, prompts, and all labeling in this document.")
```

topic	label	description	proposition
3	topic 3 (PLACEHOLDER)	placeholder description	the policy direction of topic 3 (PLACEHOLDER)
5	topic 5 (PLACEHOLDER)	placeholder description	the policy direction of topic 5 (PLACEHOLDER)
4	topic 4 (PLACEHOLDER)	placeholder description	the policy direction of topic 4 (PLACEHOLDER)

Table 1: Selected topics: the analyst-edited lookup driving strata, prompts, and all labeling in this document.

Scope: the paragraphs to classify

What this does: assigns each paragraph its dominant topic (or uses the column you saved in Script 1 via `st$topic_col`), keeps the paragraphs whose dominant topic is selected, optionally drops topically diffuse ones below `st$min_theta`, and reports the workload. There is no sampling: **every kept paragraph is classified**, so the results are a census of the selected-topic paragraphs and the only cost of scale is calendar, which the budget block states in quota windows and days. The sleep-and-retry loop and the incremental label store carry the run across windows unattended.

```
theta_mat <- para_theta |>
  dplyr::select(dplyr::all_of(theta_cols)) |>
  as.matrix()

scope_paras <- para_theta |>
  dplyr::mutate(
    stratum   = if (!is.null(st$topic_col)) .data[[st$topic_col]]
                else max.col(theta_mat, ties.method = "first"),
    max_theta = theta_mat[cbind(dplyr::row_number(), stratum)]) |>
  dplyr::filter(stratum %in% sel, max_theta >= st$min_theta) |>
  dplyr::select(para_uid, atom_id, para_id, Year, n_tokens, text,
                stratum, max_theta)

scope_paras |>
  dplyr::count(stratum, name = "paragraphs") |>
```

```
  dplyr::left_join(topics_meta |> dplyr::select(stratum = topic, label),
                   by = "stratum") |>
  dplyr::relocate(stratum, label) |>
  knitr::kable(caption = "Paragraphs to classify per selected topic (dominant-
theta assignment, after the min_theta scope rule if set).")
```

stratum	label	paragraphs
3	topic 3 (PLACEHOLDER)	1069
4	topic 4 (PLACEHOLDER)	987
5	topic 5 (PLACEHOLDER)	1035

Table 2: Paragraphs to classify per selected topic (dominant-theta assignment, after the min_theta scope rule if set).

```
to_classify <- scope_paras |> dplyr::distinct(para_uid, stratum, Year, text)
n_calls_per_model <- nrow(to_classify)
n_windows <- ceiling(n_calls_per_model / st$calls_per_window)
cat("Paragraphs to classify:", n_calls_per_model,
    "\nCalls per model:", n_calls_per_model,
    "(budget", st$calls_per_window, "per window per model; models run their",
    "windows concurrently)",
    "\nWindows:", n_windows, "-> about", round(n_windows * 4 / 24, 1),
    "days unattended with production retry settings\n")
```

```
Paragraphs to classify: 3091
Calls per model: 3091 (budget 300 per window per model; models run their windows
concurrently)
Windows: 11 -> about 1.8 days unattended with production retry settings
```

The measurement instrument, stated in full

Nothing about the prompt is hidden: it is assembled by `build_stance_prompt()` from six named components, and the assembled text for the first selected topic is printed below exactly as the model receives it. The components and their reasons:

1. **Persona** (`st$persona`, analyst-supplied): the role framing. It is part of the measurement record; changing it is changing the instrument, so it lives in config, not in code.
2. **Construct definition**: stance is position toward the stated proposition, explicitly separated from sentiment, because hostile tone can favor a target and praise can oppose it (Mohammad et al. 2016).
3. **The proposition, verbatim** from the topic CSV: stance has no meaning without a stated target, and the target must be quotable in the write-up.

4. **Relevance gate**: the model first decides whether the paragraph discusses the target at all; forcing a stance onto off-target text manufactures noise, and topic-assigned corpora guarantee off-target paragraphs occur (Li & Conrad 2024).
5. **Closed label set and ambiguity rule**: structured output (`type_enum`) makes any answer outside the four labels impossible, and genuinely ambiguous text is labeled with low confidence rather than invented certainty.
6. **Blindness rule**: judge only the text shown. The prompt never carries `Year`, ids, theta, or weights, so the instrument cannot infer the trend it is measuring.

```
cat("```\n",
    build_stance_prompt(
      proposition = topics_meta$proposition[[1]],
      persona     = st$persona),
    "\n```\n", sep = "")
```

```
You are a nonpartisan public-comment analyst.
TASK: measure STANCE, the author's position TOWARD THE STATED PROPOSITION. Stance
is not sentiment: an angry tone can still FAVOR the proposition, and warm praise
can OPPOSE it. Judge the position toward the target, not the mood.
PROPOSITION (the stance target): the policy direction of topic 3 (PLACEHOLDER)
RELEVANCE GATE: first decide whether the text discusses the target at all; if
it does not, the label is irrelevant.
LABELS: exactly one of favor, neutral, oppose, irrelevant.
AMBIGUITY: if the text is genuinely ambiguous, pick the closest label and set
confidence to low rather than inventing certainty.
BLINDNESS: judge only the text shown; ignore metadata and outside knowledge.
```

SIMULATED MODE (flip st$simulated = FALSE at work, or delete this section)

Every simulation-specific object lives here, and the code after this section carries no mode branches: downstream differences are data-driven (the presence of the planted `true_label` column). While active, this section plants a known stance trend per stratum, and **overrides `run_rater()`**, the seam defined in the source file, with an API-free fake in which each model reports the planted truth with probability 0.85. The estimates at the end then overlay the planted curves, and the gold metrics should read as a noisy-but-decent annotator by construction; if they do not, the plumbing is wrong.

```
if (st$simulated) {
  set.seed(st$seed)
  slopes <- set_names(seq(-0.012, 0.012, length.out = length(sel)), sel)
  to_classify <- to_classify |>
    dplyr::mutate(
      p_oppose = pmin(0.85, pmax(0.05,
                  0.35 + slopes[as.character(stratum)] * (Year - 2000))),
```

```
      true_label = purrr::map_chr(p_oppose, function(p)
        sample(stance_labels, 1,
               prob = c(0.9 * (1 - p) * 0.6, 0.9 * (1 - p) * 0.4,
                        p, 0.1 * (1 - p)))))

  run_rater <- function(paras, proposition, model_string, model_name,
                        persona, retries, retry_wait) {
    offset <- strtoi(substr(rlang::hash(model_name), 1, 6), base = 16L)
    set.seed(st$seed + offset)
    paras |>
      dplyr::transmute(
        id = para_uid, model = model_name, text = text,
        label = purrr::map_chr(true_label, function(tl)
          if (runif(1) < 0.85) tl else sample(setdiff(stance_labels, tl), 1)),
        confidence = sample(c("low", "medium", "high"), dplyr::n(),
                            replace = TRUE, prob = c(0.15, 0.35, 0.50)),
        rationale  = "simulated response")
  }
}
```

Classification

What this does: classifies each paragraph once per model, sequentially (one request at a time, matching how the work API is called), topic by topic so every prompt carries that topic's proposition. The label store `stance_labels.parquet` is **incremental**: paragraphs already labeled there are never re-sent, so an interrupted, repeated, or later-extended render (say, adding a topic to `st$selected_topics`) classifies only what is new and reuses every existing label, since a temperature-0 label is a fixed measurement of a fixed paragraph. Within a run, one checkpoint per (topic, model) gives resume-on-crash; failed calls sleep and retry per the config, which is how a multi-window census crosses quota resets unattended.

```
labels_path <- p_out("stance_labels.parquet")
already <- if (file.exists(labels_path)) {
  arrow::read_parquet(labels_path)
} else {
  tibble(id = character(), model = character(), text = character(),
         label = character(), confidence = character(),
         rationale = character())
}

todo <- to_classify |> dplyr::filter(!para_uid %in% already$id)

new_labels <- if (nrow(todo) == 0) NULL else {
  todo |>
    dplyr::group_split(stratum) |>
    purrr::map(function(paras_h) {
```

```
      s_id   <- dplyr::first(paras_h$stratum)
      prop_h <- topics_meta$proposition[topics_meta$topic == s_id]
      purrr::imap(st$models, function(model_string, model_name) {
        cache_rds(p_out(sprintf("stance_run%03d_s%02d_%s.rds",
                                nrow(todo), s_id, model_name)), {
          run_rater(paras_h, prop_h, model_string, model_name,
                    st$persona, st$retries, st$retry_wait)
        })
      }) |>
        purrr::list_rbind()
    }) |>
    purrr::list_rbind()
}

stance_raw <- dplyr::bind_rows(already, new_labels) |>
  dplyr::distinct(id, model, .keep_all = TRUE)
arrow::write_parquet(stance_raw, labels_path)
cat("Labels on file:", dplyr::n_distinct(stance_raw$id), "paragraphs;",
    nrow(todo), "newly classified this render.\n")
```

```
Labels on file: 3091 paragraphs; 3091 newly classified this render.
```

Reliability, consensus, triage, validity

Agreement across models is the reliability evidence; disagreement concentrates on implicitly worded text, the same text human coders disagree on, so it is information, not noise (Li & Conrad 2024). Kappa is reported with raw agreement and the per-class confusion because kappa alone has no validated interpretive thresholds. The paragraph-level point estimate is the majority vote with deterministic ties; non-unanimous items go to the human coder. Validity comes from a blind stratified random subsample: per-class precision, recall, F1, and macro-F1 (the headline under class imbalance) against hand codes; in simulated mode the planted truth stands in for the coder, closing the loop.

```
agree <- model_agreement(stance_raw)
cat("Inter-model agreement over", agree$n_items, "paragraphs:",
    "\n  kappa =", round(agree$kappa, 3),
    "   raw agreement =", round(agree$raw_agreement, 3), "\n")
```

```
Inter-model agreement over 3091 paragraphs:
  kappa = 0.594   raw agreement = 0.71
```

```
agree$confusion
```

```
            second
maverick     favor irrelevant neutral oppose
  favor        763         70     105    100
  irrelevant    65        154      41     66
  neutral       85         38     491     81
  oppose        93         62      91    786
```

```
consensus <- consensus_label(stance_raw)
triage    <- triage_for_review(stance_raw, consensus)
readr::write_csv(triage, p_out("triage_for_review.csv"))
cat(nrow(triage), "non-unanimous paragraphs written to triage_for_review.csv\n")
```

```
897 non-unanimous paragraphs written to triage_for_review.csv
```

```
set.seed(st$seed)
n_gold_h <- ceiling(st$n_gold / length(sel))
gold_ids <- to_classify |>
  dplyr::group_by(stratum) |>
  dplyr::mutate(.draw = sample(dplyr::n())) |>
  dplyr::filter(.draw <= n_gold_h) |>
  dplyr::ungroup() |>
  dplyr::pull(para_uid)

to_classify |>
  dplyr::filter(para_uid %in% gold_ids) |>
  dplyr::select(id = para_uid, text) |>
  readr::write_csv(p_out("gold_to_code.csv"))

gold <- if ("true_label" %in% names(to_classify)) {
  to_classify |>
    dplyr::filter(para_uid %in% gold_ids) |>
    dplyr::select(id = para_uid, gold_label = true_label)
} else if (file.exists(p_out("gold_coded.csv"))) {
  readr::read_csv(p_out("gold_coded.csv"), show_col_types = FALSE)
} else NULL

if (!is.null(gold)) {
  purrr::iwalk(st$models, function(model_string, model_name) {
    v <- validate_against_gold(stance_raw, gold, model_name)
    cat("\n[", model_name, "] macro-F1 =", round(v$macro_f1, 3),
        " accuracy =", round(v$accuracy, 3),
        " kappa vs gold =", round(v$kappa, 3), "\n")
    print(v$per_class |>
            dplyr::mutate(dplyr::across(c(precision, recall, f1),
                                        ~ round(.x, 3))))
```

```
  })
} else {
  cat("No gold labels yet: hand-code gold_to_code.csv, save as",
      "gold_coded.csv (columns id, gold_label), re-render.",
      "Until then the stance labels are unvalidated.\n")
}
```

```
[ maverick ] macro-F1 = 0.836  accuracy = 0.85  kappa vs gold = 0.789
# A tibble: 4 × 5
  class      precision recall    f1 support
  <chr>          <dbl>  <dbl> <dbl>   <int>
1 favor          0.909  0.8   0.851      25
2 irrelevant     0.625  1     0.769       5
3 neutral        0.786  0.846 0.815      13
4 oppose         0.938  0.882 0.909      17

[ second ] macro-F1 = 0.792  accuracy = 0.85  kappa vs gold = 0.786
# A tibble: 4 × 5
  class      precision recall    f1 support
  <chr>          <dbl>  <dbl> <dbl>   <int>
1 favor          0.957  0.88  0.917      25
2 irrelevant     0.5    0.6   0.545       5
3 neutral        0.8    0.923 0.857      13
4 oppose         0.875  0.824 0.848      17
```

Estimation: corpus proportions of stance by topic and era

What this does and how to read it: the consensus label is tabulated over the census of selected-topic paragraphs, by topic and by topic and 5-year era. Because every in-scope paragraph is classified, these are **exact descriptive quantities of that corpus**: there is no sampling and therefore no sampling error, so no confidence interval belongs on them, and putting one there would claim an uncertainty structure the design does not have. The uncertainty that does exist is **measurement error**, the LLM misclassification quantified per class by the gold metrics above, and any write-up sentence should pair a proportion with the relevant class's validated error rate. Misclassification pulls proportions toward the majority classes, so read levels jointly with the gold confusion; trends are more robust when error rates are stable across the corpus. In simulated mode the plot overlays the planted truth, and the vertical gap is exactly that attenuation, by construction.

```
est <- scope_paras |>
  dplyr::inner_join(consensus, by = c("para_uid" = "id")) |>
  dplyr::left_join(topics_meta |> dplyr::select(stratum = topic, label),
                   by = "stratum") |>
  dplyr::mutate(era = str_c(floor((Year - 2000) / st$era_width) *
                              st$era_width + 2000, "s"))
```

```
stance_by_topic <- est |>
  dplyr::count(label, consensus) |>
  dplyr::group_by(label) |>
  dplyr::mutate(p = n / sum(n)) |>
  dplyr::ungroup()

oppose_by_era <- est |>
  dplyr::group_by(label, era) |>
  dplyr::summarise(n_paras  = dplyr::n(),
                   p_oppose = mean(consensus == "oppose"),
                   .groups  = "drop")

stance_by_topic |>
  dplyr::mutate(p = round(p, 3)) |>
  tidyr::pivot_wider(id_cols = label, names_from = consensus,
                     values_from = p, values_fill = 0) |>
   knitr::kable(caption = "Stance distribution by topic over the census of
selected-topic paragraphs. Exact corpus proportions; read levels jointly with
the per-class gold error rates.")
```

label	favor	irrelevant	neutral	oppose
topic 3 (PLACEHOLDER)	0.488	0.121	0.252	0.139
topic 4 (PLACEHOLDER)	0.330	0.112	0.184	0.373
topic 5 (PLACEHOLDER)	0.418	0.117	0.205	0.260

Table 3: Stance distribution by topic over the census of selected-topic paragraphs. Exact corpus proportions; read levels jointly with the per-class gold error rates.

```
p_era <- oppose_by_era |>
  ggplot(aes(era, p_oppose, color = label, group = label)) +
  geom_line(linewidth = 0.8) +
  geom_point(aes(size = n_paras), alpha = 0.7) +
  scale_color_viridis_d(end = 0.85, name = "topic") +
  scale_size_continuous(name = "paragraphs", range = c(1, 4)) +
  labs(x = NULL, y = "P(oppose)",
       title = "Stance toward the topic proposition by era")

if ("true_label" %in% names(to_classify)) {
  truth <- to_classify |>
    dplyr::mutate(era = str_c(floor((Year - 2000) / st$era_width) *
                              st$era_width + 2000, "s")) |>
    dplyr::group_by(stratum, era) |>
    dplyr::summarise(p_true = mean(p_oppose), .groups = "drop") |>
    dplyr::left_join(topics_meta |> dplyr::select(stratum = topic, label),
                     by = "stratum")
```

```
  p_era <- p_era +
    geom_line(data = truth, aes(era, p_true, color = label, group = label),
              linetype = 2, linewidth = 0.5, show.legend = FALSE)
}
print(p_era)
```

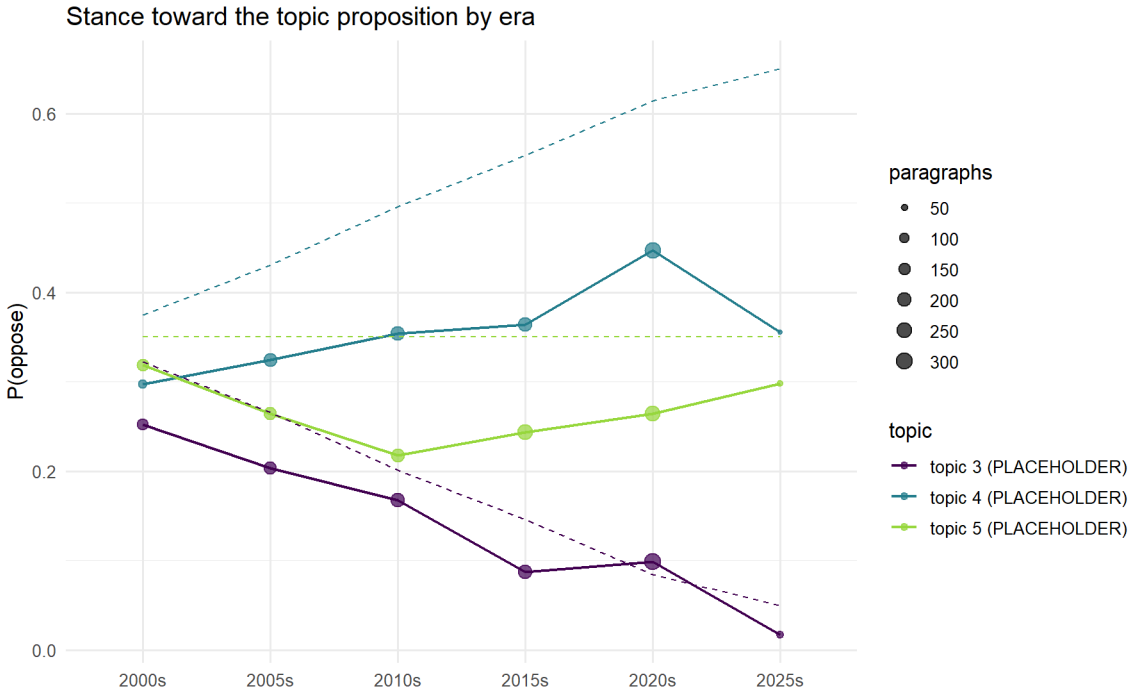

Figure 1: Share of paragraphs opposing the topic proposition, by topic and era (census tabulation of the consensus label). Point size shows paragraphs per cell. Simulated mode overlays the planted truth (dashed); the vertical gap is measurement attenuation, quantified by the gold metrics.

Handoff and measurement record

```
arrow::write_parquet(stance_by_topic, p_out("stance_by_topic.parquet"))
arrow::write_parquet(oppose_by_era,   p_out("oppose_by_era.parquet"))
cat("outputs written to", st$out_dir, ":\n",
    "- stance_labels.parquet      incremental label store, paragraph x model\n",
    "- stance_run*_s*_*.rds         per-(run, topic, model) checkpoints\n",
    "- triage_for_review.csv, gold_to_code.csv\n",
    "- stance_by_topic.parquet, oppose_by_era.parquet\n",
    "Record: models", str_c(st$models, collapse = " | "),
    "; persona:", st$persona,
    "; temperature 0; run date", format(Sys.Date()), "\n")
```

```
outputs written to outputs_stance :
 - stance_labels.parquet       incremental label store, paragraph x model
 - stance_run*_s*_*.rds        per-(run, topic, model) checkpoints
 - triage_for_review.csv, gold_to_code.csv
 - stance_by_topic.parquet, oppose_by_era.parquet
 Record: models openrouter/meta-llama/llama-4-maverick | openrouter/CHANGE-ME ;
persona: You are a nonpartisan public-comment analyst. ; temperature 0; run
date 2026-07-11
```

The defensibility record: the construct is stance toward the stated proposition in the analyst's topic CSV; the measurement covers a census of the selected-topic paragraphs (dominant-theta assignment, scope rule documented), so proportions are exact for that corpus and carry no sampling intervals; the uncertainty statement is the per-class gold error rates, with reliability from the inter-model agreement and non-unanimous items sent to human review; and the persona, prompts, model tags, temperature, and run date are pinned in this document.